Compiling Structured Operational Semantics to Executable OCaml Code

Linglong Meng, meng0181@umn.edu
University of Minnesota, Twin Cities

Introduction

Programming language semantics are traditionally written on paper as Structured Operational Semantics (SOS), then manually implemented separately. This gap between specification and implementation can introduce inconsistencies. We present a compiler extension[1] to *Sterling*[2] that bridges this gap by translating SOS specifications directly to executable OCaml code.

SOS specifications are relation-based inference rules that translate naturally to logic programming languages like Prolog. The conclusion of an inference rule (below the line) becomes a Prolog clause's head, and the premises (above the line) become the clause body. Prolog's backtracking, unification, and nondeterministic rule selection handle these relational semantics well.

However, OCaml is a functional language without these features. Our OCaml generation can only handle deterministic rules, and we need to write inference rules in a specific way with constraints. Thus, our approach works with a deterministic subset of Sterling SOS specifications. Sterling also translates SOS to Prolog, Extensibella, λProlog, and LaTeX as part of the Modular Metatheory for Extensible Languages project[3].

Challenge

One key feature of **Prolog** is **unification**, which allows implicit equality constraints, as illustrated by the following inference rule:

```
1 eval_a E A1 I
2 eval_a E A2 I
3 ------------------------- [E-EqTrue-Prolog]
4 eval_b E eq(A1, A2) trueVal
```

In Prolog, the two occurrences of `I` are automatically unified:

```
eval_b(E, eq(A1, A2), trueVal) :-
    eval_a(E, A1, I), eval_a(E, A2, I).
```

However, OCaml requires explicit equality checking. If we naively translate lines 1-2 as function calls storing results into the same variable `I`, the second assignment would simply shadow the first without performing any comparison. The correct OCaml specification must introduce distinct variables and an explicit equality check:

```
1 eval_a E A1 I1
2 eval_a E A2 I2
3 I1 = I2
4 ------------------------- [E-EqTrue-OCaml]
5 eval_b E eq(A1, A2) trueVal
```

Line 3 serves as a **control flow guard**, ensuring `I1` and `I2` are equal before the rule succeeds.

Prolog's **Backtracking** mechanism is closely connected to unification, allowing it to explore multiple derivation paths and derive unknown values in any position within a clause. For example, consider the query `eval_b [] and(trueVal, falseVal) I O` against the AND rules. Prolog will attempt the first rule [E-AndFalse1], then based on the shape matching, the base rule at line 7 fails, then tries line 10, which also fails, so it automatically backtracks to try [E-AndFalse2], ultimately deriving `I = falseVal, O = ...`. OCaml's pattern matching follows a waterfall control flow without automatic backtracking, except we are considering using DFS or other search algorithms (which may be useful for nondeterministic rules). When translating SOS rules to OCaml, we must explicitly encode the rule selection logic that Prolog handles implicitly through search.

Our Approach

Our ongoing project turns relational judgments into functional style, currently supporting a subset of Sterling SOS specifications for dynamic rules. The idea is to annotate relations to identify which argument is the primary component using a primary component marker (*). Arguments before the primary component are inputs, and those after are outputs. The primary component serves as the key discriminator for rule selection. We also group premises together using tuple pattern matching in OCaml to preserve the conjunctive semantics of Prolog clause bodies, where all premises must succeed together.

The following describes the key ideas of our approach, as a walkthrough of the E-AND compilation example into OCaml on the right side. It is not a complete description of the compiler.

First, Sterling has a basic unit called a *judgment*, which has two parts: the declaration and application. For a judgment declaration `Judgment eval_a : [(string, value)] a* value output`, we can write Prolog queries `eval_a E A I O`. Prolog can unify any argument bidirectionally that each can be a variable to infer or a concrete value to match. However, in OCaml we enforce that inputs are always concrete values, the primary component is always a constructor for distinguishing rule selection, and arguments after the primary are outputs that can be variables or concrete values. This input/primary/output discipline allows us to avoid full unification and only do one-directional evaluation from inputs to outputs, turning relational judgments into functions.

Second, we group rules by their primary component constructors and generate nested pattern matching in OCaml. For any query (function application), we first find the judgment name, then find the group by matching the primary component constructor, and finally choose the correct rule by matching the premises. Overall, the primary component plays a major role in rule selection.

After the query matches a primary component constructor's match block as shown in OCaml line 7, the input arguments are used in premises for computing outputs for the conclusion's return values, or serve as conditions for rule selection. A conditional premise is one where the premise judgment's outputs are concrete constructors, not all free variables. The user needs to group conditional premises together to do a group match combined in tuples on the function evaluation with output values like in OCaml lines 12 and 17. This simulates `Prem_1` ∧ `Prem_2` ∧ `Prem_3` → `Conclusion` in Prolog's logic.

However, this is not full backtracking as Prolog does. If the inference rules are nondeterministic, our compiler can only choose one behavior, which is the first matched rule in the same primary component group. Thus, you should write deterministic specs for OCaml generation.

Citation

[1] OCaml Backend Extension for Sterling: `https://github.com/tardism/sterling/tree/ocaml`
[2] Sterling: `https://github.com/RandomActsOfGrammar/sterling`
[3] Dawn Michaelson, Gopalan Nadathur, and Eric Van Wyk. 2025. A Modular Approach to Metatheoretic Reasoning for Extensible Languages. *ACM Trans. Program. Lang. Syst.* 47, 3, Article 10 (September 2025). `https://doi.org/10.1145/3731555`

Minimal Sterling SOS: AND Expression

```
1 Judgment eval_b:[(string,value)] b* value output
2 Judgment appendOutput : output output* output
3 b ::= and(b, b)
4     | true
5     | false
6 ------------------- [E-True]
7 eval_b E true trueVal emptyOutput
8
9 ---------------------- [E-False]
10 eval_b E false falseVal emptyOutput
11
12 eval_b E B1 falseVal O
13 -------------------------- [E-AndFalse1]
14 eval_b E and(B1, B2) falseVal O
15
16 eval_b E B1 trueVal O1
17 eval_b E B2 falseVal O2
18 appendOutput O1 O2 O
19 -------------------------- [E-AndFalse2]
20 eval_b E and(B1, B2) falseVal O
21
22 eval_b E B1 trueVal O1
23 eval_b E B2 trueVal O2
24 appendOutput O1 O2 O
25 -------------------------- [E-AndTrue]
26 eval_b E and(B1, B2) trueVal O
```

OCaml Translation

```
1  let rec eval_b e var309 =
2  (*var309 is a fresh variable for the primary
     component to be matched; e is concrete*)
3   match e , var309 with
4   ...
5    | (_, True) -> trueVal emptyOutput
6    | (_, False) -> falseVal emptyOutput
7    | (_, And (b1, b2)) ->
8      (match eval_b e b1 with
9       | (FalseVal, o) ->
10        (FalseVal, o)  (*E-AndFalse1*)
11      | _ ->
12        (match (eval_b e b1, eval_b e b2) with
13        | ((TrueVal, o1), (FalseVal, o2)) ->
14          let o = appendOutput o1 o2 in
15          (FalseVal, o) (*E-AndFalse2*)
16        | _ ->
17          (match (eval_b e b1, eval_b e b2)
              with
18          | ((TrueVal, o1), (TrueVal, o2)) ->
19            let o = appendOutput o1 o2 in
20            (TrueVal, o) (*E-AndTrue*)
21          | _ -> raise (Failure "Unmatched
              pattern"))))
```

LaTeX TikZposter